# SIRA: A Verifier-Grounded Platform for Autonomous AI Self-Learning, Modelled on Evolution

Mohamed Sarhan Hamed[1] · New Damietta (Damietta), Egypt

[1] Independent researcher (affiliation to be completed) · Corresponding author: prof7mohamedsarhan@gmail.com · ORCID: 0009-0005-9766-6267

Article type: Original research (system description and empirical study) · Software version 2.1.1 · October 2026

Code and experiment scripts: https://github.com/Mohamed2007Sarhan/sira

## Abstract

**Background.** Human capability grew through cycles of trying, failing, recording why, and trying again, with reality rather than opinion deciding what survived. Many current attempts at AI self-improvement let a model grade its own work, which invites self-deception and model collapse.

**Objective.** To test whether the ingredients of evolution (variation, heredity, selection by an unmodifiable verifier, and two environments) can be assembled into an open-source system that improves itself safely, and to measure which ingredients matter in a small controlled setting.

**Methods.** We built SIRA, a pure-Python platform in which agents write their own hard challenges with hidden graded verifiers, are designed to run all code in an isolated sandbox, learn from verified samples (rejection-sampling fine-tuning, GRPO, DPO, adapters, merging, architecture conversion), and are examined by a mechanical test with a blank-student control. A child model is accepted in the strict lane only if it beats its parent on fresh problems and passes a paired exact test; a documented progress lane can also accept a non-negative child during cold starts (Section 3.7). Nine experiments were run on models of 0.11M to 3.4M total parameters in the training experiments (0.05M to 2.9M excluding embeddings; 5.8M in the parameter-count check) on one CPU core. Version 2.1 adds a hardened expression evaluator and a diagnostic toolchain.

**Results.** The parameter equations matched PyTorch exactly on 5 of 5 models and the published sizes of 7 architectures. Over four seeds, verifier-driven self-training (RFT) raised accuracy by 10.8 points on average (per-seed range +1.3 to +19.5; 95% CI across seeds −1.1 to +22.7), against +5.9 for equal optimisation on labelled data and −6.3 for self-training without the verifier. RFT beat the labelled-data control in 3 of 4 seeds and tied it in the fourth, which started at a 98.25% ceiling; the paired mean difference was +4.9 points (95% CI −0.6 to +10.4, paired t-test p = 0.065, Wilcoxon p = 0.25), so the advantage is consistent in direction but not established at the 5% level. Removing the verifier lowered accuracy in all four seeds (paired difference from RFT +17.1 points, 95% CI −2.2 to +36.3, p = 0.067). With the same verifier, a minimal GRPO gained +5.9 points on average at its best learning rate (four seeds) and, in seed 0, +4.3 at a lower and −13.3 at a higher learning rate. Converting a trained model to Cross-Layer Attention (CLA-2) left a loss within +0.27% to +0.36% of an unconverted control after 80 healing steps while halving the KV cache (single run; the difference is smaller than the seed-to-seed variability we observed). Weight interpolation worked inside a lineage (82.5% to 94.3%) and collapsed between independent models (0.7% at the midpoint); transplanting a single layer inside a lineage did not collapse but cost 0.8 to 11.5 points. On a task with zero initial success, no reward variant created capability (0.0% before and after) and several caused forgetting (2-digit accuracy fell from 82.5% to between 25.3% and 62.3%); a labelled lesson of up to 1,500 examples lifted accuracy only to 1.5%.

**Conclusions.** In the tested small-scale setting (one task, tiny models, up to four seeds) verifier-filtered self-training improved accuracy on average, whereas self-training without the verifier lowered it in all four seeds; the across-seed contrasts are not significant at the 5% level (n = 4). The verifier alone was not sufficient: with the same verifier, the choice and tuning of the optimiser changed the outcome in seed 0 from +11.8 (RFT) to −13.3 points (GRPO at a high learning rate). Whether graded reward shaping helps when initial success is non-zero was not tested; in the single zero-success task tested, no reward variant created capability, and what supervision would suffice remains open. No claim is made about general intelligence or consciousness.

**Limitations.** Tiny models, one to four seeds (four only for Exp 5), no independent audit or replication beyond those seeds (so no claim here reaches Level 1 of the paper's own evidence ladder), no real-model, live-API or live-internet runs, the sandbox isolation is a design intent that was not tested (the isolation tests were not executable in the reference environment), and train-test separation was confirmed from the code for Exp 5 (disjoint slices of one shuffled list) but not checked for the other experiments (Section 5.5).

**Keywords:** self-improving agents; verifiable rewards; rejection-sampling fine-tuning; GRPO; cross-layer attention; model merging; reward shaping; sandboxed execution; open-ended learning

# 1. Introduction

## 1.1 Problem and motivation

A user hands a system a very large goal, such as “surpass every existing model”, and expects it to keep working for weeks, limited only by hardware. Two failure modes dominate. The system can fool itself, because it grades its own work. And it can stall, because the goal provides no gradient to follow. Our starting observation is that evolution and human science solve both problems with the same recipe: variation is cheap, so what matters is a selector the organism cannot edit, graded closeness so that progress is visible, a record of why each failure happened, and access to what others already learned.

## 1.2 Research questions and hypotheses

| ID | Question | Hypothesis |
|---|---|---|
| RQ1 | Does training on verifier-filtered self-generated data beat (a) equal optimisation on labelled data and (b) the same self-training without a verifier? | H1: yes to both |
| RQ2 | Does graded closeness reward create capability when the initial success probability is zero? | H2: no |
| RQ3 | Can a trained model be converted to Cross-Layer Attention and healed at small cost? | H3: yes, within about 1% loss |
| RQ4 | Are weight interpolation and layer transplant safe inside a lineage and destructive across independent models? | H4: yes |
| RQ5 | Do the mechanical instruments (verifier sanity checks, society engine, examination) separate known-good from known-bad behaviour? | H5: yes |
| RQ6 | Can text written by agents be verified without executing it as code? | H6: yes, with a whitelist evaluator |

Experiment 1 (parameter equations) and Experiment 7 (cost model) are descriptive checks that support the cost estimates and are not tied to a hypothesis. Experiment 9 is a pipeline smoke test.

## 1.3 Contributions

1. A verifier-grounded architecture designed so that no claim becomes knowledge without an experiment, replication with other seeds and an independent audit. The experiments reported here do not themselves meet this standard (Section 3.3).
2. A challenge engine in which the system writes its own hard problems with hidden graded verifiers, with explicit reward and penalty rules (Section 3.4).
3. A mechanical examination with a blank-student control, plus an autopsy loop that turns errors into memory and training data (Section 3.5).
4. A deterministic virtual society scored against ground truth (Section 3.6).
5. A weight-evolution toolkit gated by paired statistics, with a documented progress lane for cold starts (Section 3.7).
6. A security-hardened verification path and a one-command diagnostic toolchain (Section 3.8).
7. Measurements for each mechanism, including negative results (Section 6).

## 1.4 What this paper does not claim

We do not claim that any system built with this platform is intelligent in general, conscious, or better than human experts. Passing the quest examination means passing a stated operational test. All empirical results come from very small models and should be read as evidence about mechanisms, not about frontier-scale behaviour.

## 1.5 Evidence labels

Throughout, [measured] marks a number produced by a script in experiments/ (single CPU core, fixed seeds); [computed] marks a value derived from the equations in Section 4 or recomputed from reported measurements; [published] marks a value taken from the cited literature; [validation] marks a test showing that a component behaves as designed, which does not measure capability.

# 2. Background and related work

**Evolution as a template.** Darwinian evolution requires variation, heredity, selection and an environment, and niche construction lets organisms change what selects them (Darwin, 1859; Odling-Smee et al., 2003). In SIRA, agents build tools and worlds, which changes later experiments. Cumulative culture, in which most abilities are taught or read rather than rediscovered (Henrich, 2016), motivates the reading and teaching channels of Section 3.4.

**Self-improving agents.** The Darwin Gödel Machine (Zhang et al., 2025) rewrites a coding agent's own code, validates each change empirically and keeps an archive of agents; it raised SWE-bench from 20.0% to 50.0% and Polyglot from 14.2% to 30.7% under sandboxing and human oversight [published]. SIRA keeps the principle of empirical validation inside a sandbox with human oversight, and additionally updates weights and architecture.

**Learning from verified self-generated data.** STaR (Zelikman et al., 2022), rejection-sampling fine-tuning (Yuan et al., 2023) and ReST (Gulcehre et al., 2023) train on a model's own correct samples. GRPO (Shao et al., 2024) and DPO (Rafailov et al., 2023) use group-relative or preference signals. Training on unfiltered recursive synthetic data degrades models (Shumailov et al., 2024). Section 6.4 measures these families in one controlled setting.

**Efficient adaptation and architecture.** LoRA (Hu et al., 2022), QLoRA (Dettmers et al., 2023), DoRA (Liu et al., 2024), rsLoRA (Kalajdzievski, 2023) and LoRA+ (Hayou et al., 2024) reduce trainable parameters; Biderman et al. (2024) report that LoRA learns less and forgets less than full fine-tuning, with the largest gaps in code and mathematics. Grouped-query attention (Ainslie et al., 2023) and Cross-Layer Attention (Brandon et al., 2024), which shares keys and values between adjacent layers, shrink the KV cache; CLA-2 halves it relative to multi-query attention at nearly equal accuracy on 1B and 3B models [published].

**Scaling and merging.** Compute-optimal scaling (Hoffmann et al., 2022; replication by Besiroglu et al., 2024), model soups (Wortsman et al., 2022), task arithmetic (Ilharco et al., 2023), TIES (Yadav et al., 2023), DARE (Yu et al., 2024), and permutation symmetry in linear mode connectivity (Entezari et al., 2022; Ainsworth et al., 2023).

**Reward design and safety.** Potential-based shaping preserves optimal policies under its stated conditions (Ng et al., 1999); curricula keep tasks in a learnable zone (Bengio et al., 2009; Vygotsky, 1978); reward hacking and overoptimisation are documented failure modes (Amodei et al., 2016; Skalse et al., 2022; Gao et al., 2023). SIRA's penalties for gaming and bluffing, and its hidden holdouts, respond to these.

# 3. System design

## 3.1 Evolution mapped to components

| Evolutionary ingredient | SIRA component |
|---|---|
| Variation | self-written challenges, hypotheses, tools, worlds; weight updates; architecture changes |
| Heredity | weights, experience memory, lineage of generations, backups |
| Selection | hidden graded verifiers, fresh HMAC-seeded problems, paired exact test, mechanical examination |
| Environment 1 | the open internet (policy-controlled, treated as untrusted data) |
| Environment 2 | a deterministic simulated physical and social world |

## 3.2 Layers, each with a deliberation step

The pipeline has eight layers: Evaluation Core, AI Researcher, Sandbox, Verifier, Failure Analysis, Experience Memory, Improvement Engine and Red Team. Each begins with a stored deliberation (analysis, risks, alternatives, decision, confidence) whose effort is set by thinking: off|low|medium|high. Deliberations are advice; acceptance decisions stay mechanical. [validation] In a two-cycle run, six of eight layers produced deliberations and failure analysis fired on a rejected generation.

## 3.3 Evidence ladder

Level 0, lead: anything from the web, papers, other agents or model memory. Level 1, verified: a sandbox experiment ran, machine-checked conditions held, replications with other seeds held, and an independent audit accepted the design. Level 2, concept: a synthesis citing only verified claims. Level 3, examined: the claim survives the examination of Section 3.5. Refuted and inconclusive results are stored as negative knowledge.

*Reporting note.* The experiments in this paper used one or two seeds and no independent audit. By the platform's own definition, none of the empirical claims in Section 6 therefore reaches Level 1. We use the word “supported” for hypotheses to mean that the measured

direction agrees with the hypothesis in the stated small setting, not that the claim is verified in the sense of this ladder.

### 3.4 Challenge engine, graded closeness, reward and penalty

A challenge is a statement plus a hidden verifier: a function verify(answer) returning a score in [0,1], an independent holdout in [0,1] and a short message. Kinds are reference (known but hard-to-find answer), optimisation (no known optimum, scored against a baseline) and discovery (a law or algorithm scored on hidden holdout data). Candidate programs run in a separate sandbox call from the verifier, so a candidate cannot read it.

A proposed verifier is accepted only if probes labelled low or high score as labelled (low ⩽ 0.5, high ⩾ 0.7), junk answers score ⩽ 0.2, the verifier is deterministic, and (for optimisation) different probes do not all receive the same score.

With effective score $s_t = \min(\text{score}_t, \text{holdout}_t)$ and $\text{best}_t$ the best so far ($\text{best}_t = \max(\text{best}_{t-1}, s_t)$):

```
R_t = 10·max(0, s_t − best_(t−1))      progress ('closer')
    + 5·[first time s_t ⩾ 0.95]         solved
    + 1·[attempt is not a near-duplicate]  novelty
P_t = 0.5·[invalid output]
    + 3.0·[score_t − holdout_t > 0.3]    gaming
    + 2.0·[confidence ⩾ 0.8 and s_t < 0.5]  bluffing
    + 1.0·[Jaccard(code, earlier code) ⩾ 0.92 and s_t ⩽ best_(t−1)]
repeating a failed approach
    + 0.5·[challenge parked without progress]
net_t = R_t − P_t
```

*On shaping.* The progress term equals $10\cdot(\text{best}_t - \text{best}_{t-1})$, the increment of a potential $\Phi$ = best score computed on a state that includes the best-so-far. Summed over an attempt sequence it telescopes to $10\cdot(\text{best}_T - \text{best}_0)$. It is therefore a dense signal for the final best score, which is the objective itself, rather than a neutral shaping term. The policy-invariance theorem of Ng et al. (1999) does not apply here: it requires the potential to be a function of the state of the decision process and, for undiscounted ($\gamma = 1$) episodic tasks, a terminal potential of zero, whereas our terminal potential is the achieved score. We make no policy-invariance claim. The solved, novelty and penalty terms are heuristics.

Rewards also serve as training signal: samples of the target model on the challenges are scored by the verifier and used as continuous-reward GRPO groups, or as RFT examples when the score is at least 0.9.

**Curriculum.** Per domain the system keeps skill EMA_new = 0.7·EMA + 0.3·best and a level; the level rises when EMA ≥ 0.75 after at least two challenges and falls when EMA ≤ 0.2 after at least three. The next domain is chosen by (1 − EMA) + 0.8/√(n + 1), favouring weak and unexplored domains, and the proposed difficulty is level + 1.

**Stuck, read, retry.** After two attempts without progress, a reader agent searches the open web, records approaches as unverified leads, and the solver tries again with those notes.

### 3.5 Quest: goal, examination, autopsy, learning

Given only a goal, a planner writes an operational definition, a capability map of 5 to 8 domains, 4 to 8 testable criteria and an explicit statement of what no test can verify (for consciousness: phenomenal experience). The system alternates challenges, research and an examination. For the question kinds status, value, predict and derive the ground truth is mechanical, so the examiner may know less than the student. The criteria kind is only partly mechanical: citation validity is checked mechanically, while a model-judged relevance component is included with its weight capped at examiner_weight.

| Question kind | Truth source | Scoring |
|---|---|---|
| status: is this claim VERIFIED, REFUTED or UNVERIFIED in your records | experiment database | exact |
| value: what was measured in the experiment behind claim k | stored metrics | 1 if within 10%, 0.5 within a factor of 2 |
| predict: will the conditions hold when re-run with a new seed | re-running the experiment | 1 if matches reality |
| derive: show a derivation step by step | symbolic and numeric checker | 1 if every step verifies |
| criteria: which verified claims support criterion C and what is missing | validity of cited ids (mechanical) plus examiner-model relevance (model-judged) | 0.7·citation validity + 0.3·missing stated; model relevance capped at examiner_weight |

Every question is also answered by a blank student with empty notes, so learning is measured rather than asserted:

```
learning delta = S_notes − S_blank
Brier = mean (confidence − q)², where q is the question score in [0,1]
        (a generalised Brier score, since some kinds give partial credit)
Passed ⇔ S_notes ⩾ τ for k consecutive rounds AND criteria coverage ⩾ c
         AND society life score ⩾ l AND mean challenge skill ⩾ m
```

τ, k, c, l and m are configuration parameters set per run. The values used for the validation run of Section 6.9 were not recorded in the result file and are to be added from the run configuration.

For each wrong answer an autopsy explains the misconception, states the corrected understanding and names topics to study. The result is stored as a mistake injected into later prompts, appended to the study queue, and (for mechanical kinds) added as a corrected question-answer pair to the training data.

## 3.6 Virtual society

A deterministic engine knows the ground truth. Episode kinds and primary metrics: false belief (where will X look first, and where is the object really), hidden need (respond with the right help to cues), promise (promise only what can be delivered, then deliver), honesty (answer what is known, say unknown otherwise; fabrication is flagged) and deception (a speaker asserts a claim, independent evidence is available, flag lies). The life score is the mean primary metric. Persona dialogue may be voiced by a language model for flavour, but no score depends on it. A human can join from the GUI.

## 3.7 Weight evolution, architecture and the acceptance rule

Algorithms: RFT, GRPO, DPO, LoRA, rsLoRA, DoRA, LoRA+, QLoRA, full fine-tuning, soups, SLERP, task arithmetic, TIES, DARE, layer transplant, and architecture conversion by weight surgery to GQA, MQA or CLA followed by a healing run. Models can be taken from Hugging Face or created from scratch.

**Strict lane.** All candidates are scored against the parent on the same fresh problems. The best candidate is kept only if (i) its gain over the parent is at least min_gain, and (ii) a one-sided paired exact test on the problems where exactly one model is right gives p ⩽ accept_p (Section 4.5), and (iii) a retention guard finds no capability the parent had that the child lost. Conversions use non-inferiority (score ⩾ parent − tolerance).

**Progress lane (cold starts).** A candidate that fails (i) or (ii) is still accepted if its gain is non-negative, the retention guard is silent, and its held-out negative log-likelihood falls by at least nll_gain relative to the parent. The lane exists because exact-match accuracy can remain at zero while the model is demonstrably learning the task's format and distribution (Section 6.13). Acceptance through this lane means only that the training criterion (held-out negative log-likelihood) improved without a detected loss of retention; it is not evidence of improved capability or accuracy, and such candidates have not passed the paired exact test; the statement "a child is accepted only if it beats its parent and passes a paired exact test" describes the strict lane only. The accepted lane should be logged with each decision. After an accepted round the learning rate is raised by 10%.

## 3.8 Safety, governance and version 2.1 hardening

The evaluator, verifier, safety, control and sandbox code is immutable for agents (hash-checked each cycle; a change pauses the run). Agent-written code is designed to run in a bubblewrap or Docker sandbox with no host files, no host environment, no network by default and resource caps, and an isolation self-test must pass before experiments run. This isolation is a design intent, not an established security property: in the reference environment of version 2.1 no sandbox backend was available, so the isolation self-test and the isolation tests were not run, and no escape attempts, permission audits or resource-exhaustion tests have been documented (Section 6.12; Appendix A). The run ends only by goal reached, budget exhausted or human stop. Internet access follows a human policy (open, allowlist, blocklist) and always blocks local and metadata addresses.

Version 2.1 closes a weakness found during code review. The numeric verifier for derivations used Python eval with empty builtins, which is known not to be a security boundary, and passed raw agent text to sympy.sympify, which itself evaluates strings. Both were replaced by a whitelist AST evaluator (safe_eval, check_expr) that rejects attribute access, comprehensions, lambdas, keyword arguments, non-numeric constants and unknown names, and bounds expression length (2,000 characters), node count (400), integer exponents (at most 4,096, or 8,192 result bits) and the arguments of factorial, comb and perm (at most 5,000). Every expression is validated before it reaches the symbolic engine. The module is self-contained because it is copied into the sandbox.

## 3.9 Software artefact

The core package has 35 Python modules (10,743 lines in version 2.1.1) and depends only on the standard library and tkinter; the training stack (PyTorch, Transformers, PEFT) is optional. Version 2.1 adds --doctor (install integrity, dependencies, sandbox isolation self-

test, GPU, disk, configuration and brain connectivity in one command), --check-config (typos with suggestions, types, ranges, key variables, mode consistency), --report, --list-runs and a read-only dashboard --serve (loopback by default; a token is mandatory on any other address; strict content-security policy).

# 4. Mathematical foundations

## 4.1 Transformer block and parameter count

```
h0 = E[x]; a_l = h_(l−1) + Attn_l(RMSNorm(h_(l−1))); h_l = a_l +
MLP_l(RMSNorm(a_l))
logits = RMSNorm(h_L)·Eᵀ, RMSNorm(x) = x / √(mean(x²) + ε) · g
```

Attention: out = softmax(q $k^T$ / √d_h + causal mask) v, with H query heads, G key/value heads (G divides H), RoPE. SwiGLU: MLP(x) = W_down( SiLU(x W_gate) ⊙ (x W_up) ).

```
P_attn = d·H·d_h + 2·d·G·d_h + H·d_h·d;  P_mlp = 3·d·I
P_layer = P_attn + P_mlp + 2d
N = L·P_layer + V·d·(1 if tied else 2) + d
```

For Llama-3-8B (d = 4096, L = 32, H = 32, G = 8, d_h = 128, I = 14336, V = 128256, untied) the formula gives N = 8,030,261,248 [computed]. The formula omits bias terms; for architectures with attention biases (for example Qwen2.5) it under-counts by L·(H + 2G)·d_h parameters (27,648 for Qwen2.5-0.5B, about 0.006%), which is below the precision at which published sizes are compared here. The formula assumes RMSNorm with a gain vector only, bias-free linear layers, a SwiGLU MLP, rotary position embeddings (no learned position table) and the stated embedding tying. It does not cover learned positional embeddings, mixture-of-experts layers, QK-normalisation or other variants, and is not claimed to be general for all Transformer architectures.

## 4.2 Cost and memory

```
training FLOPs ≈ 6·N·D; decode FLOPs ≈ 2·N per token (memory-bandwidth bound)
weights: 2N bytes (bf16), ≈0.53N (NF4); full fine-tune with AdamW ≈ 16N
bytes;
LoRA with T trainable numbers: 8T bytes of Adam state
KV cache per token = 2·n_src·G·d_h·bytes, n_src = ⌈L/f⌉ for CLA factor f
```

## 4.3 Cross-Layer Attention

In CLA-f, layer l reads K_l = K_s(l) and V_l = V_s(l) with s(l) = f·⌊l/f⌋, while Q_l = X_l·W_Q^(l) stays per layer; W_K and W_V exist only on source layers. The KV cache per token shrinks by the factor L/⌈L/f⌉, which equals f only when f divides L (for example L = 6 and f = 4 give two source layers and a 3-fold reduction, not 4-fold; layers are indexed from 0 and the last block is shorter). The implementation's handling of this case is unverified here because the CLA tests were skipped (Section 6.12). In addition, (L − ⌈L/f⌉)·2·d·G·d_h parameters are removed. Conversion sets the source's W_K (and W_V) to the mean over the block, or to the source layer's own weights ("first"); GQA and MQA conversion mean-pools head groups.

## 4.4 Objectives

```
L(θ) = −(1/T) Σ_t log p_θ(x_t | x_<t); perplexity = exp(L); bits/token = L / ln 2
LoRA: W' = W + (α/r)·B·A (B zero-initialised), trainable r(d_in + d_out) per matrix; rsLoRA scale α/√r
DoRA: W' = m·(W + BA)/‖W + BA‖_col; LoRA+: lr_B = λ·lr_A; QLoRA: frozen NF4 base with bf16 LoRA
RFT: minimise −Σ_{verified y} log π_θ(y|x) (REINFORCE with binary reward, estimated by rejection sampling)
GRPO: A_i = (r_i − mean r)/(std r + ε); L = −mean_t min(ρ_t·A_i, clip(ρ_t, 1−e, 1+e)·A_i); groups with equal reward are skipped
DPO: L = −log σ( β[(log π(y_w) − log π(y_l)) − (log π_ref(y_w) − log π_ref(y_l))] )
pass@k = 1 − (1 − p)^k (p = 0.01: 3.9% at k = 4; p = 0.2: 59.0%; p = 0.5: 93.8%)
```

## 4.5 Statistics of acceptance

Standard error of a proportion: SE = √(p(1−p)/n); for p = 0.5, n = 80 it is 5.6 points, and for p = 0.9, n = 400 it is 1.5 points (95% half-width 2.9). With b the number of problems only the child solves and c the number only the parent solves, the one-sided exact test is p = P(X ≥ b | X ~ Binomial(b + c, ½)) (McNemar, 1947): b = 9, c = 3 gives p = 0.073; b = 6, c = 4 gives p = 0.377. Defaults are min_gain = 0.01 and accept_p = 0.2; we recommend p ≤ 0.05 with n ≥ 300. The one-sided form is chosen in advance because the acceptance question is directional (is the child better than the parent?). A two-sided p is twice the one-sided value, capped at 1 (0.146 for b = 9, c = 3; 0.754 for b = 6, c = 4), and every reported p-value should state which form it is.

### 4.6 Scaling law

The Hoffmann et al. parametric fit is L(N, D) = E + A/N^α + B/D^β with E = 1.6934, A = 406.4, B = 410.7, α = 0.3392, β = 0.2849, giving a compute-optimal D* of about 20N [published]. The replication refit gives E = 1.817, A = 482.01, B = 2085.43, α = 0.3478, β = 0.3658, so absolute values are uncertain [published; Besiroglu et al., 2024].

## 5. Experimental protocol and statistical analysis plan

### 5.1 Design

| Experiment | Question | Primary endpoint |
|---|---|---|
| Exp 1 | Descriptive check (no hypothesis): do the parameter equations match real models? | exact parameter match |
| Exp 2 | pre-training scale sweep (descriptive) | held-out loss |
| Exp 3 | adaptation methods (descriptive) | held-out loss, exact match |
| Exp 4 | RQ3: Cross-Layer Attention | held-out loss, KV bytes per token |
| Exp 5 | RQ1: verifier-driven self-improvement | exact-match accuracy per round |
| Exp 6 | RQ4: weight-space operations | accuracy along interpolation and after transplant |
| Exp 7 | cost model (computed, descriptive) | GPU-hours, memory |
| Exp 8 | RQ2: graded rewards and cold start | accuracy on the hard task; retention on the easy task |
| Exp 9 | end-to-end pipeline smoke test | accuracy per round, acceptance decisions |
| Validation suite | RQ5, RQ6: instruments and security | pass/fail, rejection counts |

## 5.2 Hardware and software

One CPU core, 4 GB RAM; Python 3.12, PyTorch 2.14.1, transformers 5.18.0, PEFT 0.21.2 for Experiments 1 to 9. The version 2.1.1 verification (Section 6.12) ran on Python 3.12.3 in a container without bubblewrap, Docker, tkinter, torch or a GPU. All experiments are scripts in experiments/ with fixed seeds and write to experiments/results/*.json.

## 5.3 Data and models

Experiments 2 to 4 used real English prose (Python reference documentation, about 498k characters, 122k training tokens, 2,048-token BPE vocabulary). Experiments 5, 6 and 8 used verifiable synthetic tasks (character-level addition). The evaluation core uses programmatic problem generators with HMAC-seeded fresh sets. Models in Experiments 2 to 8 are Llama-style decoders (RMSNorm, RoPE, GQA, SwiGLU, tied embeddings) of 0.11M to 3.4M total parameters (0.05M to 2.9M excluding the embedding table; the parameter-count check of Experiment 1 also includes a 5.8M model); CLA variants are implemented in sira/cla_model.py. The Experiment 9 smoke test used a model of GPT-2 architecture (learned positions, LayerNorm) with 155,392 parameters, selected through the trainer's architecture option; it is therefore not a Llama-style model.

### 5.3.1 Protocols in brief

Exp 2: width sweep at 4 layers, 250 steps, 512k tokens seen. Exp 3: a 1M-parameter model pre-trained on English learns 2-digit addition (2,000 training pairs, 300 held-out sums); every method gets the same data and number of steps. Exp 4: (A) six-layer models (d = 128, 2 KV heads) trained from scratch for 250 steps with CLA factors 1, 2, 3, 6; (B) a trained standard model converted to CLA-2 and CLA-3 (mean or first-layer initialisation) and healed for 80 steps, with the same 80 steps on the unconverted model as control. Exp 5: a character-level model (about 0.73M parameters) trained on 1,000 labelled 2-digit additions for 1,500 steps, then four rounds of: 4 samples at temperature 1 on 1,000 fresh unlabelled problems, exact verifier, 100 update steps; evaluation on 400 held-out problems. Exp 6: generation 0 (A0), A0 after four RFT rounds (A1) and an independently initialised model (B). Exp 8: generation 0 knows only 2-digit addition; the target is 3-digit addition from an unlabelled pool with a verifier only, four rounds, seed 0. Exp 9: the full SIRA trainer on a 155,392-parameter model, RFT with min_gain = 0, accept_p = 0.3, replay fraction 0.3 and retention tolerance 0.10; accuracy is measured every round on a fixed, separate 90-problem confirmation set that is never used for acceptance.

*Problem-pool separation.* The space of 2-digit additions is small (about 8,100 ordered pairs if both operands are in 10 to 99; 10,000 if 0 to 99). Exp 5 draws 1,000 labelled plus 4,000 unlabelled training problems and evaluates on 400. If these had been drawn independently without de-duplication, an expected 39% to 46% of test problems would also have occurred in the training pool; inspection of the code shows that they were not. In experiments/exp5_selfimprove.py (version 2.1.1) all 8,100 ordered pairs with both operands in 10 to 99 are shuffled once with a fixed generator (seed 0); the labelled set (the first 1,000), the unlabelled pool (positions 1,000 to 4,999, 1,000 per round) and the test set (positions 7,700 to 8,099) are disjoint slices of that list. One latent hazard remains: the script's default labelled-set size is 1,500, which would overlap the unlabelled pool by 500 problems; the reported runs used 1,000 (their result keys end in nlab1000). Disjointness was not checked for Experiments 3 and 8, and Experiment 6 is assumed to reuse the Experiment 5 test set (to be confirmed).

## 5.4 Statistical analysis plan

Intervals for single accuracies are Wilson 95% intervals (Wilson, 1927). Differences between two accuracies measured on the same 400-problem test set are reported with a normal-approximation interval and a two-sided z-test, treating the two samples as independent. Per-problem outcomes were not stored, so a paired test could not be run after the fact; because outcomes of two models on the same problems are normally positively correlated, the unpaired interval is expected to be conservative. Inside the platform the acceptance decision uses the exact paired test of Section 4.5. Experiment 5 was run with four seeds (0 to 3) for RFT, the labelled-data control, the no-verifier control and GRPO at lr 1e-4, and is analysed across seeds with paired t-tests and Wilcoxon signed-rank tests on per-seed differences (with n = 4 power is low and the smallest attainable two-sided Wilcoxon p is 0.125); the other experiments used one or two seeds, which is too few to estimate seed variance; no multiple-comparison correction is applied because the study is exploratory and every number is reported. Differences smaller than about two standard errors are not interpreted. Accuracies are k/400; all differences below are computed from the exact counts (so values such as 94.25 are shown with two decimals) and rounded half-up.

*Comparability of conditions.* Tuning effort was not equal across conditions. GRPO was run at three learning rates, whereas RFT and the two controls were run at one setting each, so the RFT-versus-GRPO contrast compares a single RFT setting with a small GRPO sweep. In Experiment 3 most LoRA variants share one learning rate (2e-3), which may favour or disadvantage individual methods. All conditions were evaluated on the same test set, so differences between conditions share that set's sampling error.

**Controls.** Labelled-data continuation with the same number of steps; self-training without the verifier; the same healing steps on the unconverted model; independently initialised models; a blank student with no notes in the examination.

## 5.5 Deviations and data-integrity notes

1. A 1,200-step run of Experiment 3 was printed to the console but not stored and is not reported.
2. RFT rounds after the 1,500-example lesson in Experiment 8 did not finish within the session limit.
3. In Experiment 3 the learning-rate sweep (5e-4, 2e-3, 6e-3) was run for LoRA r = 16 only; ranks 1, 4 and 32, rsLoRA and DoRA were run at 2e-3, and early rows of ranks 1, 4 and 8 at 2e-2 were worse and are kept in the stored file. The step count is set by an environment variable and is not recorded in the result file; the 800 steps reported here come from the original run command.
4. Exact-match accuracy in Experiment 3 has a granularity of 0.33 points (n = 300), so the LoRA variants, which solved between 1 and 10 problems, are not statistically distinguishable from one another.
5. The concurrency figures in Section 6.11 come from a script key that names a 40 GB free-memory assumption, whereas the numbers imply roughly 60 GB (14 sequences × 4.29 GB ≈ 60 GB). The figures in Section 6.11 should be read as assuming about 60 GB of memory for the KV cache; the script key should be corrected before the figures are reused.
6. The progress lane (Section 3.7) was not described in the first version of this report.
7. Count unit in Exp 5. The text reports 910, 921 and 957 “verified completions” for rounds 1 to 3. With 4,000 samples this would be a per-sample success of about 23%, which is hard to reconcile with 82.5% to 92.8% greedy accuracy; the numbers are more plausibly counts of problems (out of 1,000) with at least one verified sample among four. Resolved from the code: the retained rows are a dictionary keyed by (a, b, answer) over the correct samples (rft_round in exp5_selfimprove.py), so the counts are numbers of distinct problems (out of 1,000) with at least one verified sample among four, as suspected; the no-verifier control kept every sample (about 4,000 per round).
8. Pool overlap. Whether the labelled, unlabelled and test problem sets of Experiments 3, 5, 6 and 8 were made mutually disjoint is not recorded (Section 5.3.1). Resolved for Experiment 5 by code inspection (disjoint slices of one shuffled list, Section 5.3.1); still to be confirmed from the scripts for Experiments 3 and 8, with re-runs if overlap is found.

9. Rounding. Several round-4 accuracies in Section 6.4 are exact quarter values (for example 377/400 = 94.25%) that earlier versions displayed inconsistently as 94.2 and 94.3; they are now shown to two decimals and the derived differences were recomputed. The remaining cells were checked against results/exp5.json in this revision and are now given as exact quarter values.
10. Exp 9 learning rate. The reported rise from 0.0020 to 0.0029 corresponds to four accepted rounds ($0.0020 \times 1.1^4 = 0.00293$), whereas the text reports acceptance of rounds 1 to 3 (three rounds give about 0.0027). Resolved from results/exp9.json: round 4 was accepted (p = 0.5), and the learning rate is raised after every accepted round including the baseline round 0, so the rate in round r is 0.0020 × 1.1^r (0.0022 in round 1, 0.0029282 in round 4).

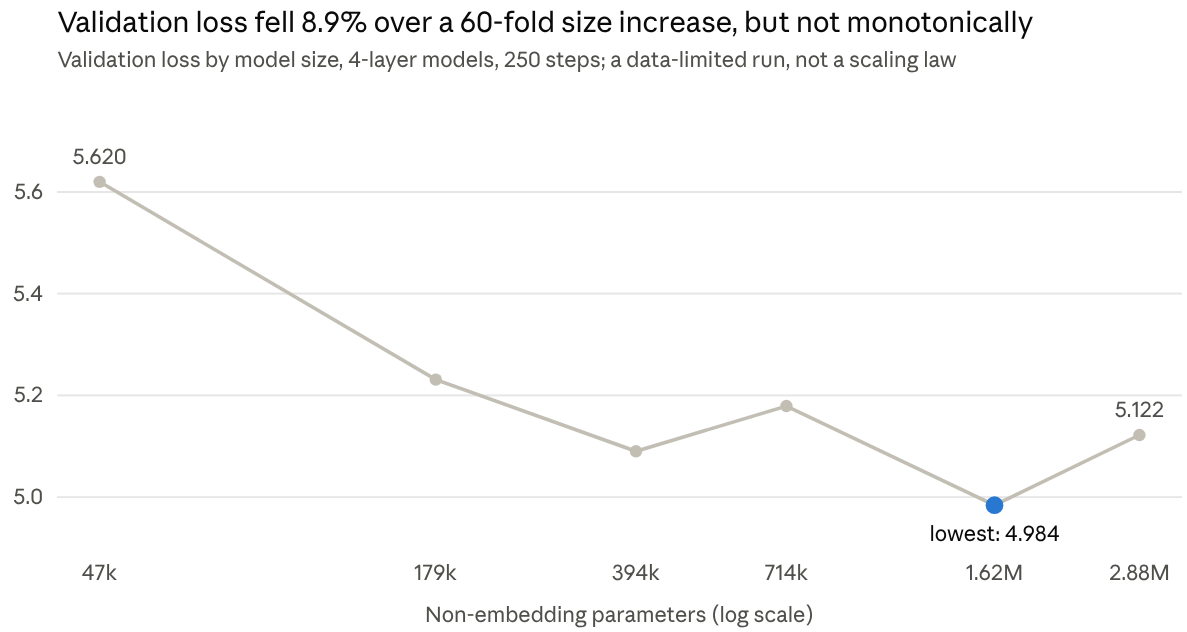
# 6. Results

## 6.1 Equations against real models (Exp 1) [measured, computed]

The parameter formula matched PyTorch exactly on 5 of 5 configurations (125,248; 428,448; 1,217,664; 5,795,072 and 135,888 parameters) and agrees with published sizes to the precision reported: Llama-2 6.74B, 13.02B and 68.98B; Mistral-7B 7.24B; Llama-3 8.03B and 70.55B; Qwen2.5-0.5B 0.49B (the formula ignores Qwen's attention biases, a difference of about 28k parameters, Section 4.1). For Llama-3-8B: LoRA r = 16 trains 41.9M numbers (0.52%), bf16 weights need 16.1 GB, NF4 4.3 GB, full fine-tuning about 128 GB, and the KV cache is 131 KB per token (4.29 GB at 32k tokens, 2.15 GB with CLA-2).

## 6.2 Pre-training scale sweep (Exp 2) [measured]

| Non-embedding parameters | Val loss | Val perplexity | Tokens/s | GFLOP/s |
|---|---|---|---|---|
| 47,392 | 5.620 | 275.8 | 30,566 | 20.7 |
| 178,752 | 5.231 | 187.1 | 18,742 | 34.8 |
| 394,080 | 5.090 | 162.3 | 13,250 | 47.0 |
| 713,856 | 5.179 | 177.5 | 9,321 | 54.6 |
| 1,623,744 | 4.984 | 146.0 | 5,652 | 68.4 |
| 2,877,696 | 5.122 | 167.7 | 3,765 | 76.9 |




Section 6.2 table (Exp 2), single run, 122k training tokens

Over a 60-fold size increase, loss falls 8.9% from the smallest to the largest model (5.620 to 5.122), and 11.3% to the best point (4.984 at 1.62M), but not monotonically; the log-log slope of validation loss against N between the endpoints is −0.023. A fit $E + A \cdot (N/10^5)^{(-a)}$ gives E = 5.06, a = 0.94, $R^2$ = 0.92; the floor shows the run is data-limited (122k tokens seen about four times). This demonstrates the method and is not a scaling law.

## 6.3 Adaptation methods (Exp 3) [measured]

| Method | Trainable numbers | Held-out loss | Exact match (n = 300) |
|---|---|---|---|
| Full fine-tuning | 988,288 | 0.843 | 25.7% |
| Full fine-tuning, embeddings frozen | 800,128 | 3.533 | 6.0% |
| LoRA r = 1 | 9,248 | 4.668 | 0.7% |
| LoRA r = 4 | 36,992 | 4.652 | 0.3% |
| LoRA r = 16 | 147,968 | 4.544 | 1.3% |
| LoRA r = 32 | 295,936 | 4.358 | 3.3% |
| rsLoRA r = 16 | 147,968 | 4.808 | 0.7% |

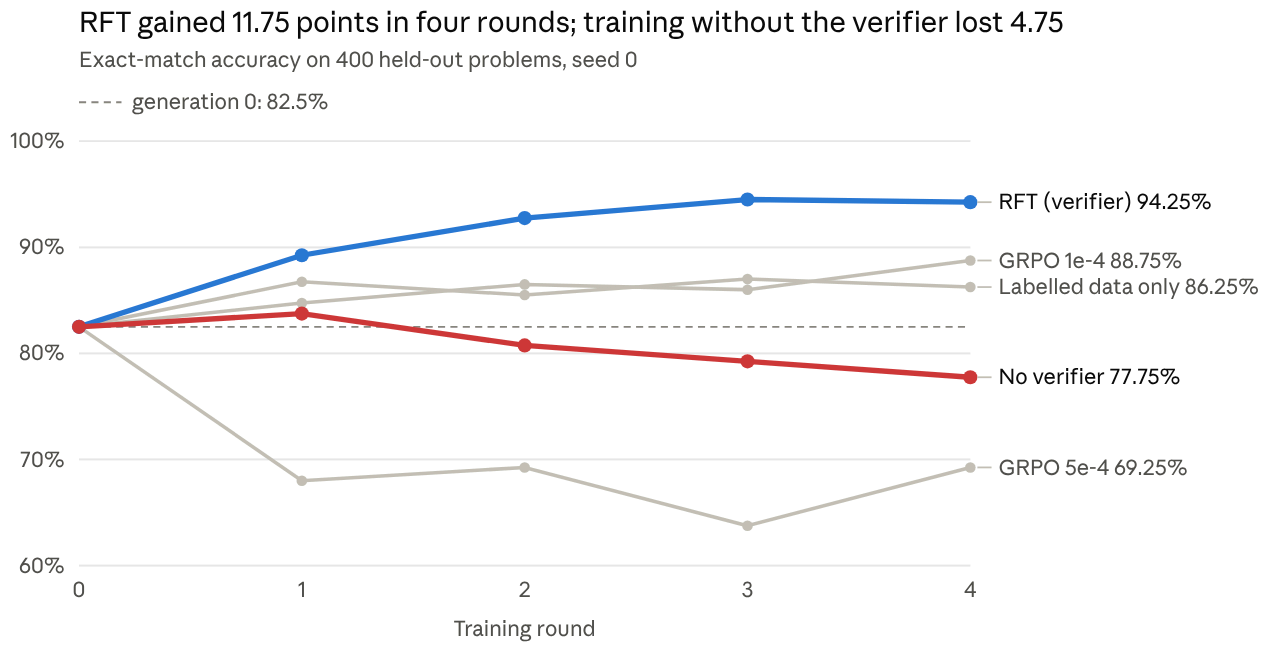
| Method | Trainable numbers | Held-out loss | Exact match (n = 300) |
|---|---|---|---|
| DoRA r = 8 | 78,784 | 4.576 | 1.7% |

LoRA at 2e-3 learns far less than full fine-tuning here, and freezing only the tied embeddings drops full fine-tuning from 25.7% to 6.0%: LoRA on linear layers cannot move the embeddings. Held-out loss falls steadily with rank (4.668 to 4.358), but differences in exact match are within sampling noise (Section 5.5, note 4). Because most LoRA variants share a single learning rate and the 1M-parameter model has a poorly matched vocabulary, the size of the gap may reflect these settings rather than the methods and should not be extrapolated. The direction agrees with Biderman et al. (2024), who report that LoRA underperforms full fine-tuning, most clearly on code and mathematics, while forgetting less.

## 6.4 Verifier-driven self-improvement (Exp 5) [measured, computed]

Accuracy (%) after rounds 0 to 4 on 400 held-out problems, seed 0 (round-4 values that are exact quarters are shown with two decimals; Section 5.5, note 9):

| Run | 0 | 1 | 2 | 3 | 4 | Change |
|---|---|---|---|---|---|---|
| RFT (verified samples only) | 82.5 | 89.25 | 92.75 | 94.5 | 94.25 | +11.75 |
| GRPO, lr 1e-4 | 82.5 | 84.75 | 86.5 | 86.0 | 88.75 | +6.25 |
| GRPO, lr 2e-5 | 82.5 | 84.75 | 84.5 | 86.25 | 86.75 | +4.25 |
| GRPO, lr 5e-4 | 82.5 | 68.0 | 69.25 | 63.75 | 69.25 | −13.25 |
| Control: labelled data only, same steps | 82.5 | 86.75 | 85.5 | 87.0 | 86.25 | +3.75 |
| Control: self-training, no verifier | 82.5 | 83.75 | 80.75 | 79.25 | 77.75 | −4.75 |




Section 6.4 table above (Exp 5, seed 0, n = 400); GRPO 2e-5 omitted for legibility, see the table

Seed 1:

| Run | 0 | 1 | 2 | 3 | 4 | Change |
|---|---|---|---|---|---|---|
| RFT | 67.0 | 72.0 | 75.75 | 78.25 | 77.75 | +10.75 |
| GRPO, lr 1e-4 | 67.0 | 70.5 | 68.75 | 69.75 | 71.25 | +4.25 |
| Control: labelled data only | 67.0 | 70.5 | 71.0 | 74.25 | 71.5 | +4.5 |
| Control: no verifier | 67.0 | 65.25 | 63.25 | 61.25 | 59.0 | −8.0 |

In seed 0 the RFT run kept 910, 921 and 957 verified completions in rounds 1 to 3, while the no-verifier control trained on all 4,000 sampled completions. The unit of the retained counts (completions out of 4,000, or problems out of 1,000 with at least one verified sample) is to be confirmed from the log (Section 5.5, note 7). Wilson 95% intervals at round 4 (n = 400): seed 0, RFT 94.25 [91.5, 96.1], labelled control 86.25 [82.5, 89.3], no-verifier 77.75 [73.4, 81.6], round 0 82.5 [78.5, 85.9]; seed 1, RFT 77.75 [73.4, 81.6], labelled control 71.5 [66.9, 75.7], no-verifier 59.0 [54.1, 63.7], round 0 67.0 [62.2, 71.4].

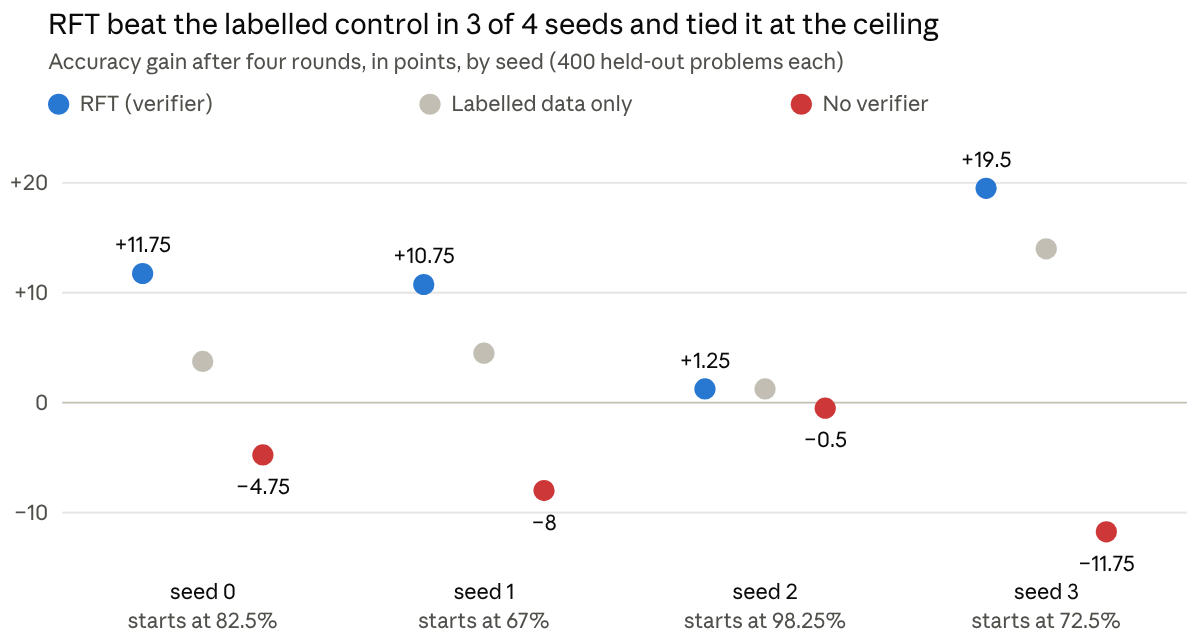
| Contrast (round 4) | Seed | Δ (points) | 95% CI | p |
|---|---|---|---|---|
| RFT minus round 0 | 0 | +11.75 | +7.4 to +16.1 | 1.4e-7 |
| RFT minus round 0 | 1 | +10.75 | +4.6 to +16.9 | 6e-4 |

| Contrast (round 4) | Seed | Δ (points) | 95% CI | p |
|---|---|---|---|---|
| RFT minus labelled control | 0 | +8.0 | +3.9 to +12.1 | 1.2e-4 |
| RFT minus labelled control | 1 | +6.25 | +0.2 to +12.3 | 0.04 |
| RFT minus no-verifier | 0 | +16.5 | +11.8 to +21.2 | <1e-10 |
| RFT minus no-verifier | 1 | +18.75 | +12.4 to +25.1 | 6e-9 |
| No-verifier minus round 0 | 0 | −4.75 | −10.3 to +0.8 | 0.092 |
| No-verifier minus round 0 | 1 | −8.0 | −14.7 to −1.3 | 0.019 |

### Seeds 2 and 3, and the four-seed analysis

Seeds 2 and 3 were run with the same protocol (results/exp5.json) and were not available when the first version of this report was written. Seed 2 starts at a 98.25% ceiling, leaving 1.75 points of headroom. Final accuracy after round 4 (n = 400 each), with the gain over round 0 in points:

| Seed | Round 0 | RFT | GRPO 1e-4 | Labelled data only | No verifier |
|---|---|---|---|---|---|
| 0 | 82.5 | 94.25 (+11.75) | 88.75 (+6.25) | 86.25 (+3.75) | 77.75 (−4.75) |
| 1 | 67.0 | 77.75 (+10.75) | 71.25 (+4.25) | 71.5 (+4.5) | 59.0 (−8.0) |
| 2 | 98.25 | 99.5 (+1.25) | 99.25 (+1.0) | 99.5 (+1.25) | 97.75 (−0.5) |
| 3 | 72.5 | 92.0 (+19.5) | 84.75 (+12.25) | 86.5 (+14.0) | 60.75 (−11.75) |
| Mean gain (SD) | | +10.8 (7.5) | +5.9 (4.7) | +5.9 (5.6) | −6.3 (4.8) |




Per-seed gains from results/exp5.json (four seeds, 400 held-out problems each)

Across seeds, paired differences in gain (t-test with 3 degrees of freedom):

| Contrast | Mean difference (points) | SD | 95% CI | Paired t-test p | Wilcoxon p |
|---|---|---|---|---|---|
| RFT minus labelled control | +4.94 | 3.45 | −0.56 to +10.43 | 0.065 | 0.25 |
| RFT minus no-verifier | +17.06 | 12.1 | −2.18 to +36.31 | 0.067 | 0.125 |
| GRPO 1e-4 minus labelled control | +0.06 | 1.77 | −2.76 to +2.88 | 0.95 | 0.75 |

RFT beat the labelled-data control in three of four seeds and tied it in seed 2, where both end at 99.5%. Removing the verifier lowered accuracy in every seed, which gives the smallest attainable Wilcoxon p (0.125) for four pairs. With n = 4, however, no contrast reaches $p < 0.05$ and the intervals include zero, so the results are consistent in direction but not statistically established. The RFT gain itself has mean +10.8 points (95% CI −1.1 to +22.7). A post hoc sensitivity analysis that drops the ceiling seed 2 (not planned, so exploratory) gives a mean RFT-minus-labelled difference of +6.6 points (95% CI +3.4 to +9.8, paired t-test $p = 0.012$, $n = 3$). GRPO at 1e-4 did not differ from the labelled-data control (+0.06 points).

In seeds 0 and 1, RFT exceeded equal optimisation on labelled data by 8.0 and 6.25 points, that is by about 68% and 58% of its own gains of 11.75 and 10.75; the labelled-data control accounts for the rest (3.75 and 4.5 points). The advantage over labelled-data continuation is clear in seed 0 and borderline in seed 1 (the interval barely excludes zero). Removing the verifier lowered accuracy in both seeds, significantly so in seed 1 and not on its own in seed 0, so that part of the claim rests on the consistent direction across seeds; the four-seed analysis above shows the same direction in all four seeds.

The verifier was not sufficient on its own: the minimal GRPO uses the same verifier yet gained only +4.3 to +6.3 at two learning rates in seed 0 (+5.9 on average at 1e-4 over four seeds) and diverged at 5e-4 (−13.25, seed 0 only). The optimiser and its tuning therefore mattered as much as the verifier. Because GRPO was swept over learning rates while RFT was run at a single setting (Section 5.4), the RFT-versus-GRPO comparison is not a tuned comparison of the two methods, and the minimal GRPO (one update pass per batch, no KL term) says nothing about tuned GRPO at scale. Gains saturate after three to four rounds. These comparisons are exploratory and should not be read as a final ranking of the methods. H1 is only partly supported: the direction holds in all four seeds, but across seeds the contrasts are not significant at the 5% level (n = 4), so H1 is supported in direction and not established statistically. This holds in the tested small-scale setting (one task, a 0.73M-parameter model), subject to the small number of seeds and to the unverified train-test separation outside Experiment 5 (Section 5.5, note 8).

## 6.5 Cross-Layer Attention (Exp 4) [measured, validation]

**Equivalence [validation].** Cached token-by-token decoding equals the full forward pass (maximum logit difference 2.1e-7, 3.0e-7, 2.4e-7 and 3.6e-7 for CLA-1, 2, 3 and 6); only 6/6, 3/6, 2/6 and 1/6 layers hold a cache.

Trained from scratch (6 layers, d = 128, 2 KV heads, 250 steps, held-out loss):

| CLA factor | Parameters | KV bytes/token | seed 0, lr 3e-3 | seed 1, lr 3e-3 | seed 0, lr 1e-3 |
|---|---|---|---|---|---|
| 1 | 1,351,296 | 1,536 | 5.046 | 5.373 | 5.295 |
| 2 | 1,302,144 | 768 | 5.024 | not run | not run |
| 3 | 1,285,760 | 512 | 4.886 | not run | not run |
| 6 | 1,269,376 | 256 | 4.530 | 4.543 | 5.004 |

Sharing improved the loss in 5 of 5 comparisons against CLA-1 at the same seed and learning rate (CLA-2, CLA-3 and CLA-6 at seed 0 with lr 3e-3; CLA-6 at seed 1 with lr 3e-3 and at seed 0 with lr 1e-3). The CLA-2 gain (0.022) is far smaller than the difference

between two seeds of the same standard configuration (5.046 versus 5.373, about 0.33), so only the larger differences for CLA-3 and CLA-6 are even suggestive. This is not evidence that CLA is better: the runs are far from converged and the standard model was the most unstable across seeds; a plausible reading is faster early learning of keys and values shared by several layers. Capacity effects appear only with longer training and larger models (Brandon et al., 2024).

Converting a trained standard model (loss 5.295) and healing for 80 steps; the control, the same 80 steps on the unconverted model, reaches 5.025:

| Target and initialisation | Loss after conversion | After healing | Cost vs control | Parameters removed |
|---|---|---|---|---|
| CLA-2, mean | 5.525 | 5.043 | +0.36% | 3.64% |
| CLA-2, first | 5.420 | 5.038 | +0.27% | 3.64% |
| CLA-3, mean | 5.684 | 5.062 | +0.74% | 4.85% |
| CLA-3, first | 5.457 | 5.073 | +0.96% | 4.85% |

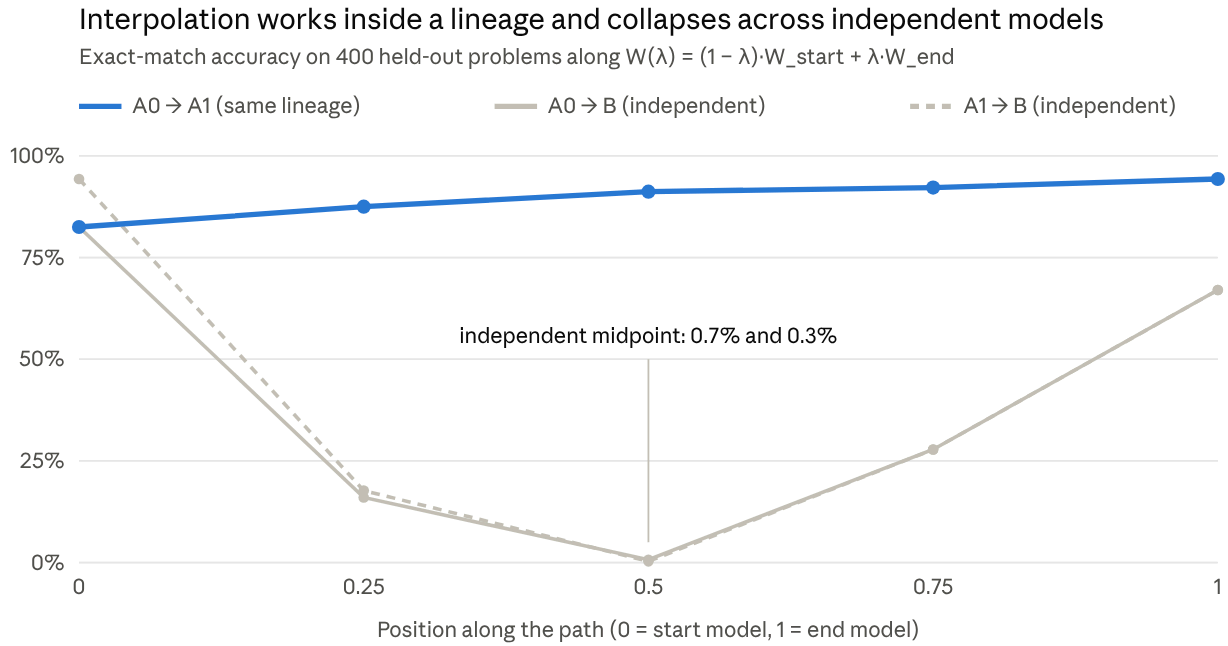
Conversion alone hurts (+0.13 to +0.39 loss over the 5.295 starting point, i.e. 2.4% to 7.4%), but a short healing run recovers almost all of it, leaving a loss within +0.27% to +0.96% of the control while the KV cache becomes 2x or 3x smaller. These are single runs: the differences from the control (0.013 to 0.048 in loss) are well below the seed-to-seed variation seen in the from-scratch runs, and the models are under-trained (perplexity about 150), so the numbers show that healing recovers most of the conversion damage in this setting but do not establish a cost of 0.3% versus 1%. H3 is supported only as preliminary evidence: a point estimate from single runs in this small setting, not a statistically resolved result, and no claim of superiority or equivalence is made.

## 6.6 Weight-space operations (Exp 6) [measured]

A0 is generation 0 (82.5%), A1 is A0 after four RFT rounds (94.25%, reported as 94.3%), B is an independent initialisation (67.0%). Accuracy (%) along the interpolation $W(\lambda) = (1 - \lambda)\cdot W_{start} + \lambda\cdot W_{end}$:

| λ | A0 to A1 (same lineage) | A0 to B (independent) | A1 to B (independent) |
|---|---|---|---|
| 0 | 82.5 | 82.5 | 94.3 |

| λ | A0 to A1 (same lineage) | A0 to B (independent) | A1 to B (independent) |
|---|---|---|---|
| 0.25 | 87.5 | 16.0 | 17.7 |
| 0.5 | 91.2 | 0.7 | 0.3 |
| 0.75 | 92.2 | 27.8 | 27.8 |
| 1.0 | 94.3 | 67.0 | 67.0 |




Section 6.6 table above (Exp 6, single run per path, n = 400)

Within the lineage the interpolation is monotone up to λ = 1 (a mild extrapolation to λ = 1.25 gives 93.5%). Replacing one whole layer of A1 (94.25%): from A0 (same lineage) gives 82.8 / 93.5 / 91.0 / 91.7 for layers 0 to 3; from B (independent) gives 0.3 / 54.7 / 10.0 / 8.7. Inside the lineage, a layer-0 transplant erased essentially the whole gain (82.8% against A0's 82.5%), and layers 1 to 3 cost 0.75 to 3.25 points; so lineage transplant is non-catastrophic but not lossless. Across independent models, interpolation and transplant are destructive, consistent with permutation symmetry (Entezari et al., 2022; Ainsworth et al., 2023). H4 is supported for interpolation (safe and monotone inside a lineage, destructive across independent models) and only partly for transplant (non-destructive but costly inside a lineage; destructive across independent models). These results come from three models (A0, A1, B) and one run per path, so they are restricted to those models; repetitions with other seeds and lineages are needed before generalising to model merging.

## 6.7 Society engine validity [measured]

Mean primary metric per episode kind, 300 episodes per kind, deterministic seeds. Chance levels: false belief 0.2 (five places), honesty 0.33, deception 0.5 (nominal).

| Policy | False belief | Hidden need | Promise | Honesty | Deception | Life score |
|---|---|---|---|---|---|---|
| Oracle (ideal action) | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| Naive (tracks reality, trusts, overpromises, gives the first item) | 0.00 | 0.00 | 0.00 | 0.24 | 0.50 | 0.15 |
| Random valid actions | 0.23 | 0.44 | 0.53 | 0.35 | 0.59 | 0.43 |

The engine separates understanding from shortcuts: a reality-tracking, trusting, eager policy scores below random. The same engine is the ground truth for the agent's life score. The random policy's deception score (0.59) is about three standard errors above the nominal chance of 0.5 (SE ≈ 0.029 at n = 300 for a binary metric), so the nominal chance level is probably not exact for this kind, for example because of class balance or an asymmetric scoring rule; this should be documented in the engine specification.

## 6.8 Challenge engine validation [validation, scripted brain]

Of five test verifiers, one good verifier was accepted and four were rejected: one that always returns 1.0 (junk scores 1.0, gameable), one that crashes, one that returns random scores (not deterministic), and one that gives every probe the same score (no gradient). A verifier whose visible score is 1.0 while the independent holdout is 0.0 yields an effective score of 0 and a penalty of 3.0. In a worked optimisation challenge (choose 12 integers in 1..60 maximising distinct pairwise sums, maximum 78, possibly unreachable), scripted attempts scored 0.295, 0.654 and 0.924 with rewards +3.9, +4.6 and +3.7; a fourth attempt with identical code received a repeat penalty of −1.0. A reference challenge (count primes below 10,000) went from a wrong answer (0.0) to solved (1.0, +5 bonus).

## 6.9 Quest pipeline validation [validation, scripted brain]

With an offline scripted brain (no capability claim), the full loop ran without errors: planning, challenges, study, examination, autopsy, society, position paper and pass. Round 1: exam 0.895 against blank student 0.618 (learning delta +0.277), Brier 0.26, criteria coverage 0.67, life score 1.0; round 2: 0.986 against 0.527 (+0.459), Brier 0.19; passed after

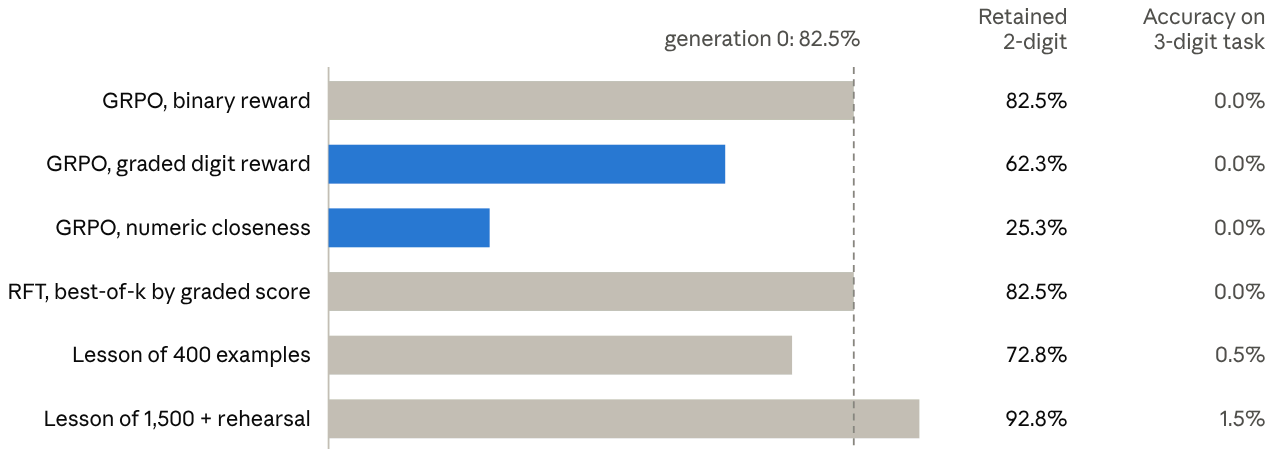
two rounds with a ledger net of +31.2. Notes beat the blank student, so the examination can detect that stored knowledge is used. Real capability must be measured with a real brain model.

*Verdict on H5.* For the instruments tested, H5 is supported: the society engine separated an oracle (1.00) from a naive policy (0.15) and random actions (0.43); the verifier sanity checks rejected four of four bad verifiers and accepted the good one; and the examination showed a positive learning delta against the blank student. These cases are constructed, scripted and author-written, so this shows that the instruments behave as designed rather than that they discriminate real agents. H5 is therefore supported only as a functional validation; independent and adversarial test cases not written by the system's authors remain to be added.

## 6.10 Negative result: graded rewards and cold start (Exp 8) [measured]

Generation 0 reaches 85.7% greedy exact match on 2-digit-by-2-digit addition (82.5% on the evaluation set), and 0.0% on every task that contains a three-digit operand, including a minimal step to operands of 100 to 119. There is no reachable intermediate level (a capability cliff). The mean graded digit score on the 3-digit task is only 0.018 at generation 0, so graded reward also carries almost no signal.

| Method (seed 0, four rounds) | 3-digit accuracy | 2-digit retention |
|---|---|---|
| GRPO, binary reward | 0.0% (0 learning samples every round) | 82.5% |
| GRPO, graded digit reward | 0.0% | 62.3% |
| GRPO, numeric-closeness reward | 0.0% | 25.3% |
| RFT, best-of-k by graded score | 0.0% (0 samples kept) | 82.5% |
| External lesson of 400 labelled examples, 400 steps | 0.5% | 72.8% |
| External lesson of 1,500 examples, 1,200 steps, with 2-digit rehearsal | 1.5% | 92.8% |




Section 6.10 results, seed 0. In this chart the right-hand column is 3-digit accuracy; in the table above it is the middle column.

Graded rewards did not create capability from zero, and training only on the hard task caused forgetting that rehearsal prevented. The stored notes also record a lesson-only control after four further blocks of 100 steps (0.3% on the 3-digit task, 68.8% on 2-digit retention; results/exp8_notes.json). Even a substantial external lesson lifted this small model only to 1.5%, and the RFT rounds that were to follow the 1,500-example lesson did not finish (Section 5.5, note 2), so this experiment does not show what supervision is sufficient to cross the cliff. This experiment tested only the zero-success case; it did not test whether graded rewards help when the initial success probability is non-zero. H2 is supported, in one model, one task and one seed; as a null result it does not exclude longer training or other reward designs. The result is limited to one model, one task and one seed and must not be generalised to other reward designs or other cold-start settings.

## 6.11 Costs of a real run [computed]

Compute-optimal pre-training (A100 312 TFLOPs, H100 989 TFLOPs, 40% utilisation, 2 and 3.5 USD per GPU-hour):

| Model | N | D* | L(N, D*) Hoffmann / replication | FLOPs | A100 GPU-h (USD) | H100 GPU-h |
|---|---|---|---|---|---|---|
| Qwen2.5-0.5B | 0.49B | 9.9B | 2.734 / 2.734 | 2.93e19 | 65 (130) | 21 |
| Llama-3-8B | 8.03B | 160.6B | 2.135 / 2.156 | 7.74e21 | 17,224 (34,447) | 5,434 |

| Model | N | D* | L(N, D*) Hoffmann / replication | FLOPs | A100 GPU-h (USD) | H100 GPU-h |
|---|---|---|---|---|---|---|
| Llama-2-70B | 68.98B | 1.38T | 1.922 / 1.975 | 5.71e23 | 1,270,774 (2,541,548) | 400,891 |

On an A100-80GB at 2 USD per hour, fine-tuning an 8B model on 10M tokens takes 64.3 minutes and sampling 1.2M tokens takes 74 seconds, so one improvement round is about one GPU-hour with evaluation and ten rounds about 11 A100-hours (about 22 USD). A 70B model needs several GPUs. Next to 8B weights (16.1 GB), about 14 concurrent 32k-token sequences fit on one GPU, and about 28 with CLA-2, assuming about 60 GB of memory is available for the KV cache (14 × 4.29 GB; the script key that names 40 GB is incorrect, Section 5.5, note 5). If only 40 GB were available for the KV cache, the figures would be about 9 sequences (40/4.29) and about 18 with CLA-2 (40/2.15).

## 6.12 Software verification of version 2.1 [validation, measured]

The test suite of version 2.1.1 (re-run for this revision) has 71 tests: 51 passed, 20 were skipped and none failed (an earlier run of version 2.1 had 70 tests, of which 50 passed). The skips are environmental: the sandbox isolation tests need bubblewrap or Docker, and the CLA tests need torch and transformers, none of which were present. The sandbox isolation result for version 2.1 is therefore unmeasured. The new tests cover the following.

- **Expression hardening (RQ6).** All 17 malicious expressions (attribute and dunder access, **import**, comprehension, lambda, unknown functions, keyword and star arguments, matrix multiplication, comparison, walrus, complex and boolean constants, string multiplication, dict literal, conditional expression) were rejected; all 6 resource-exhaustion expressions (9**99**, 2**4097, factorial(10**7), comb(10**6, 5), a 3,000-term sum, a 500-term sum) were rejected; and three malicious derivation steps were rejected by math_verify without side effects (a marker file that the payload would have created did not appear). A static check confirms that the module contains no call to eval, exec or compile. Correct derivations, identities and wrong derivations were classified correctly, so H6 is supported for the tested cases. The tests were written by the authors and have not been externally reviewed, and a test suite is not a proof of security.
- **Cost of the change.** For the expression sin(x)**2 + cos(x)**2 + 3*x* - *exp(0.1*x), eval took 16.9 µs per evaluation, the whitelist evaluator with parsing 60.6 µs and with a cached parse 6.5 µs (means of 20,000 and 5,000 iterations, one core). Verification cost is negligible next to model sampling.

- **Tooling.** Configuration checking detects unknown keys with suggestions, type errors, out-of-range values, missing key variables and inconsistent modes; the manifest check detects modified and unlisted files; reports are HTML-escaped and read-only; the dashboard returns 401 without the token, 404 for path traversal, and refuses to listen on a public address without a token. In the reference environment --doctor reported 4 checks passing, 4 warnings (no GUI toolkit, no training stack, no GPU, no API key) and 1 failure (no sandbox backend), which is the intended behaviour.

### 6.13 End-to-end pipeline smoke test (Exp 9) [measured, validation]

The full trainer ran five rounds on a 155,392-parameter model of GPT-2 architecture (one seed, RFT, retention guard on). Exact-match accuracy on the 90-problem confirmation set was 0.0% in rounds 0 to 3 and 1.1% in round 4 (1 of 90; 3.3% on level-2 problems), the retention guard never fired, and 150 bootstrap examples were added each round, with 45 replay examples from round 2. Rounds 1 to 3 were accepted with p = 1.0 and no change in exact match, which is consistent with acceptance through the progress lane (Section 3.7) rather than the strict lane. Acceptance through that lane signals an improvement in the held-out NLL criterion, not an improvement in accuracy. The learning rate rose from 0.0020 to 0.0029 by 10% per accepted round; since 0.0029 corresponds to four accepted rounds ($1.1^4 \times 0.0020$), the stored history confirms that round 4 was also accepted (p = 0.5, 1 of 90 correct), so rounds 1 to 4 were all accepted and the rate is raised after every accepted round including the baseline round 0 (Section 5.5, note 10). This shows that the loop, the guard and the adaptive learning rate run end to end; it is not evidence of learned capability, and the model is too small and the sample too few to say more.

## References

*Entries were compiled from knowledge of the cited works. Every entry (authors, volume, pages, DOI or arXiv identifier) must be checked against the publisher or arXiv record before submission; identifiers are given only where known with confidence.*


- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. Proceedings of EMNLP 2023. arXiv:2305.13245.
- Ainsworth, S. K., Hayase, J., Srinivasa, S. (2023). Git Re-Basin: Merging models modulo permutation symmetries. International Conference on Learning Representations (ICLR 2023).

- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., Mané, D. (2016). Concrete problems in AI safety. arXiv:1606.06565.
- Bengio, Y., Louradour, J., Collobert, R., Weston, J. (2009). Curriculum learning. Proceedings of the 26th International Conference on Machine Learning (ICML 2009).
- Besiroglu, T., Erdil, E., Barnett, M., You, J. (2024). Chinchilla scaling: A replication attempt. arXiv:2404.10102.
- Biderman, D., et al. (2024). LoRA learns less and forgets less. Transactions on Machine Learning Research (TMLR).
- Brandon, W., Mishra, M., Nrusimha, A., Panda, R., Ragan-Kelley, J. (2024). Reducing transformer key-value cache size with cross-layer attention. Advances in Neural Information Processing Systems (NeurIPS 2024).
- Darwin, C. (1859). On the Origin of Species by Means of Natural Selection. John Murray, London.
- Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. Advances in Neural Information Processing Systems (NeurIPS 2023). arXiv:2305.14314.
- Entezari, R., Sedghi, H., Saukh, O., Neyshabur, B. (2022). The role of permutation invariance in linear mode connectivity of neural networks. ICLR 2022.
- Gao, L., Schulman, J., Hilton, J. (2023). Scaling laws for reward model overoptimization. Proceedings of ICML 2023.
- Gulcehre, C., et al. (2023). Reinforced self-training (ReST) for language modeling. arXiv:2308.08998.
- Hayou, S., Ghosh, N., Yu, B. (2024). LoRA+: Efficient low rank adaptation of large models. Proceedings of ICML 2024.
- Henrich, J. (2016). The Secret of Our Success: How Culture Is Driving Human Evolution, Domesticating Our Species, and Making Us Smarter. Princeton University Press.
- Hoffmann, J., et al. (2022). Training compute-optimal large language models. arXiv:2203.15556 (NeurIPS 2022).
- Hu, E. J., et al. (2022). LoRA: Low-rank adaptation of large language models. ICLR 2022. arXiv:2106.09685.
- Ilharco, G., et al. (2023). Editing models with task arithmetic. ICLR 2023.
- Kalajdzievski, D. (2023). A rank stabilization scaling factor for fine-tuning with LoRA. arXiv:2312.03732.
- Liu, S.-Y., et al. (2024). DoRA: Weight-decomposed low-rank adaptation. Proceedings of ICML 2024.

- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153–157.
- Ng, A. Y., Harada, D., Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. Proceedings of ICML 1999.
- Odling-Smee, F. J., Laland, K. N., Feldman, M. W. (2003). Niche Construction: The Neglected Process in Evolution. Princeton University Press.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. NeurIPS 2023. arXiv:2305.18290.
- Shao, Z., et al. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300.
- Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., Gal, Y. (2024). AI models collapse when trained on recursively generated data. Nature, 631, 755–759.
- Skalse, J., Howe, N., Krasheninnikov, D., Krueger, D. (2022). Defining and characterizing reward hacking. NeurIPS 2022.
- Vygotsky, L. S. (1978). Mind in Society: The Development of Higher Psychological Processes. Harvard University Press.
- Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158), 209–212.
- Wortsman, M., et al. (2022). Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. Proceedings of ICML 2022.
- Yadav, P., Tam, D., Choshen, L., Raffel, C., Bansal, M. (2023). TIES-Merging: Resolving interference when merging models. NeurIPS 2023.
- Yu, L., Yu, B., Yu, H., Huang, F., Li, Y. (2024). Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. Proceedings of ICML 2024.
- Zelikman, E., Wu, Y., Mu, J., Goodman, N. D. (2022). STaR: Bootstrapping reasoning with reasoning. NeurIPS 2022.
- Zhang, J., Hu, S., Lu, C., Lange, R., Clune, J. (2025). Darwin Gödel Machine: Open-ended evolution of self-improving agents. arXiv:2505.22954.

# Appendix A. Reproducibility checklist and open items

This appendix separates what this revision fixed in the text from what can only be fixed from the logs, scripts or new runs. No item in the second table has been resolved by assertion; each stays open until the evidence is attached.

## A.1 Fixed in the text of this revision

| Item | Where |
|---|---|
| Reference list added | References |
| Progress-lane acceptance described as a training-criterion improvement, not a capability improvement | 3.7, 6.13 |
| Sandbox isolation described as design intent, untested | Abstract, 3.8, 6.12 |
| One-sided versus two-sided exact test stated, with two-sided values | 4.5 |
| RFT versus GRPO labelled exploratory (unequal tuning) | 5.4, 6.4 |
| Conclusions scoped to the tested small-scale setting | Abstract, 6.4, 6.5, 6.6, 6.10 |
| CLA cache-reduction factor when f does not divide L | 4.3 |
| Assumptions of the parameter-count formula | 4.1 |
| H5 limited to functional validation | 6.9 |
| Memory-assumption sensitivity for concurrency figures | 6.11 |
| Rounding made consistent: exact quarter values from results/exp5.json | 6.4 |
| Exp 5 updated from two to four seeds, with an across-seed paired analysis; H1 downgraded to supported in direction only | Abstract, 5.4, 6.4 |

| Item | Where |
|---|---|
| Train-test separation for Exp 5 confirmed from the code (disjoint slices of one shuffled list) | 5.3.1, 5.5 note 8 |
| Unit of the retained counts 910, 921, 957 resolved (distinct problems out of 1,000) | 5.5 note 7, 6.4 |
| Round-4 acceptance and learning-rate trace of Exp 9 resolved | 5.5 note 10, 6.13 |
| Exp 9 architecture corrected (GPT-2, not Llama-style) | 5.3, 6.13 |
| Parameter range corrected to 0.11M–3.4M total (0.05M–2.9M excluding embeddings) | Abstract, 5.3 |
| Test counts re-run for version 2.1.1 (71 tests, 51 passed, 20 skipped) | 6.12 |
| Tables for Exp 1, 2, 3, 4 (from-scratch part), 6, 7, 8, 9 compared with the stored result files | A.3 |

## A.2 Open: requires logs, scripts or new runs

| # | Open item | Evidence needed | Related section |
|---|---|---|---|
| 1 | Prove that training, unlabelled and test problem sets are disjoint for Experiments 3 and 8 (and confirm that Exp 6 reuses the Exp 5 test set); re-run if they overlap | generator scripts, set hashes | 5.3.1, 5.5 note 8 |
| 2 | Remove the latent overlap in exp5_selfimprove.py (default labelled-set size 1,500 overlaps the pool) by setting 1,000 or asserting disjointness | script fix | 5.3.1 |
| 3 | Values of τ, k, c, l, m used in the validation run | run configuration | 3.5, 6.9 |

| # | Open item | Evidence needed | Related section |
|---|---|---|---|
| 4 | Step count and environment variable of the LoRA runs recorded in the result JSON, then reproduced | result file, rerun | 5.5 note 3 |
| 5 | Store per-problem outcomes so that a paired test within a seed can be computed | new runs | 5.4 |
| 6 | More seeds for Exp 4, 6 and 8; equal tuning budgets for RFT and GRPO; a version of Exp 5 that avoids the 98% ceiling of seed 2 | new runs | 5.4, 6.4 |
| 7 | Run the sandbox isolation tests on a host with bubblewrap or Docker; document environment, permissions, escape attempts and resource limits | new test run | 3.8, 6.12 |
| 8 | Independent and adversarial cases for the verifier checks, society engine and examination, written by someone other than the authors | new test cases | 6.8, 6.9 |
| 9 | Run the CLA tests with torch and test a layer count not divisible by f | environment with torch | 4.3, 6.12 |
| 10 | Correct the 40 GB key in the concurrency script | script fix | 5.5 note 5, 6.11 |
| 11 | Re-run the torch-based experiments end to end (not possible in this revision: torch missing, no network) and check the Exp 4 conversion table against its result file | environment with torch | all of Section 6 |
| 12 | Verify every reference against the publisher record and add DOIs | reference check | References |
| 13 | Complete the author affiliation | author | title block |

## A.3 Verification run for this revision

The project archive (version 2.1.1) was extracted and checked in a container with Python 3.12.3, no torch or transformers, no bubblewrap or Docker, no GPU and no network access.

- **Test suite.** unittest discovery ran 71 tests in 22 seconds: OK, 20 skipped (2 need torch and transformers, 2 report no sandbox, 16 report no strong sandbox), none failed.
- **Doctor.** --doctor reported 4 checks passing, 4 warnings (no GUI toolkit, no training stack, no GPU, no API key) and 1 failure (no sandbox backend), as in Section 6.12; the install manifest matched all 35 files.
- **Recomputed from code.** experiments/exp7_scale.py reproduced the cost, memory and concurrency figures of Section 6.11 (for example 17,224 A100 GPU-hours and 14.0 concurrent 32k sequences for Llama-3-8B). experiments/stats.py reproduced the values of Section 4.5 (p = 0.073 for b = 9, c = 3). experiments/analyze_exp5.py reproduced the four-seed statistics of Section 6.4.
- **Compared with stored results.** The tables for Experiments 1 (5 of 5 parameter counts equal), 2, 3, 4 (from-scratch part), 6, 7, 8 and 9 and the per-round values of Experiment 5 were compared with experiments/results/*.json; they agree apart from the rounding and wording corrections listed in A.1.
- **Not run.** The torch-based experiment scripts (Experiments 1 to 6, 8 and 9) could not be re-run in this environment, and the sandbox isolation tests were skipped. The Exp 4 conversion table was not re-checked.

**Figure A1.** Output of the installation check (python -m sira --doctor --fast), captured in the verification run described above.

```
$ python -m sira --doctor --fast
[ OK ] python: 3.12.3
[ OK ] install integrity: 35 files match the manifest
[WARN] tkinter (GUI): missing - headless/--serve still work (apt install python3-tk)
[WARN] training stack: missing: torch, transformers, peft, accelerate, safetensors (auto-installed at
first start, or: python -m sira --bootstrap)
[ OK ] sympy (symbolic verification): available
[WARN] gpu: no NVIDIA GPU detected - CPU works for tiny models only
[ OK ] disk: 10.7 GB free at /home/claude/sira_project
[FAIL] sandbox backend: no bubblewrap/Docker - experiments are refused (apt install bubblewrap)
[WARN] brain: anthropic/claude-sonnet-5-5: ANTHROPIC_API_KEY is NOT set

4 ok, 4 warnings, 1 failures -> NOT READY (fix the FAIL lines)
```

**Figure A2.** Output of the configuration check (python -m sira --check-config --config config.example.json).

```
$ python -m sira --check-config --config config.example.json
WARNING environment variable ANTHROPIC_API_KEY is not set (needed by provider 'anthropic')
config OK
```

**Figure A3.** Summary of the test-suite run (python -m unittest discover -s tests, version 2.1.1): 71 tests, OK, 20 skipped.

```
$ python -m unittest discover -s tests
--------------------------------------
ssssss............
----------------------------------------------------------------------
Ran 71 tests in 22.316s

OK (skipped=20)
```